



# Week 3 Lab

## Kubernetes Observability with Prometheus & Grafana

### Lab Objective

In this lab, we will deploy Prometheus and Grafana on a local Kubernetes cluster to understand how observability works in practice.

The focus is on seeing Kubernetes behavior through metrics and dashboards, not on production configuration.

---

### What You Will Learn

By the end of this lab, you will understand:

- How Kubernetes exposes metrics natively
- How Prometheus discovers and scrapes metrics
- How Grafana visualizes metrics from Prometheus
- How cluster behavior becomes visible without SSH

- Why metrics are usually the first step in observability
- 

## Lab Scope & Disclaimer

This lab:

- Is part of Kubernetes Month
- Uses a local Kubernetes cluster
- Focuses on metrics and visualization only

This lab does not cover:

- Logs or tracing
  - Alerting or SLOs
  - Production hardening or security
- 

## Prerequisites

You need:

- A local Kubernetes cluster
  - kubectl
  - helm
- 

## Mental Model

- Kubernetes **exposes metrics**
- Prometheus **collects metrics**
- Grafana **visualizes metrics**
- Together, they help you understand what the cluster is doing over time

Simple rule:

| Prometheus collects. Grafana explains.

---

## Step 1: Prepare the Environment

Add the Prometheus Helm repository:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

Create a namespace for observability:

```
kubectl create namespace monitoring
```

---

## Step 2: Install Prometheus + Grafana (kube-prometheus-stack)

We will install a single stack that includes:

- Prometheus
- Grafana
- Kubernetes metric exporters

We explicitly set a Grafana password to avoid login issues during workshops.

```
helm install kps prometheus-community/kube-prometheus-stack \
  -n monitoring \
  --set grafana.adminPassword=admin123
```

Wait until all pods are running:

```
kubectl get pods -n monitoring
```

Do not continue until Grafana and Prometheus are in `Running` state.

---

## Step 3: Access Prometheus

Port-forward Prometheus:

```
kubectl port-forward -n monitoring svc/kps-kube-prometheus-stack-prometheus 9090:9090
```

Open:

```
http://localhost:9090
```

## Verify Prometheus Is Working

- Go to **Status** → **Targets**
- Confirm targets are **UP**

Run a simple query:

```
up
```

This shows which components are currently reachable.

## Step 4: Access Grafana

Port-forward Grafana:

```
kubectl port-forward -n monitoring svc/kps-grafana 3000:80
```

Open:

```
http://localhost:3000
```

Login:

- **Username:** `admin`
- **Password:** `admin123`

## Step 5: Verify Prometheus Is Connected to Grafana

In Grafana:

1. Go to **Connections** → **Data sources**
2. Open **Prometheus**
3. Confirm status is **Connected**

This proves Grafana is reading metrics from Prometheus.

---

## Step 6: Explore Kubernetes Dashboards

In Grafana:

- Go to **Dashboards** → **Browse**
- Open Kubernetes dashboards such as:
  - Cluster overview
  - Nodes
  - Pods

Focus on:

- CPU usage
- Memory usage
- Pod count
- Node health

Teaching point:

| You don't need to understand every graph — just what "normal" looks like.

---

## Step 7: Deploy a Demo Application

Create a demo namespace and deploy nginx:

```
kubectl create namespace demo
kubectl -n demo create deployment web --image=nginx
```

```
kubectl -n demo expose deployment web --port=80
kubectl get pods -n demo
```

At this stage, the app is running but idle.

---

## Step 8: Generate Load and Observe Metrics

Generate traffic inside the cluster:

```
kubectl -n demo run load --rm -it --image=curlimages/curl --r
estart=Never -- \
  sh -c "for i in $(seq 1 300); do curl -s http://web > /dev/
null; done; echo done"
```

Observe in Grafana:

- CPU usage increases
- Resource graphs move
- Metrics change in real time

This is observability in action.

---

## Step 9: Scale the Application

Scale the deployment:

```
kubectl -n demo scale deployment web --replicas=3
kubectl -n demo get pods
```

Observe in Grafana:

- Pod count increases
  - Resource usage adjusts automatically
  - Kubernetes behavior becomes visible
-

## Step 10: Simulate Failure

Delete a pod:

```
kubectl -n demo delete pod -l app=web
```

Observe:

- Kubernetes recreates the pod
- Dashboards show disruption and recovery
- No manual intervention is needed

This demonstrates why observability is critical in dynamic systems.

---

## Key Takeaways

- Kubernetes already exposes powerful signals
  - Prometheus collects metrics using a pull model
  - Grafana turns metrics into understanding
  - Metrics show **trends and symptoms**
  - Observability removes the need for guesswork
- 

## Cleanup

```
helm uninstall kps -n monitoring  
kubectl delete ns monitoring demo
```